Algoritmo de *Work Stealing*

Octubre/2026



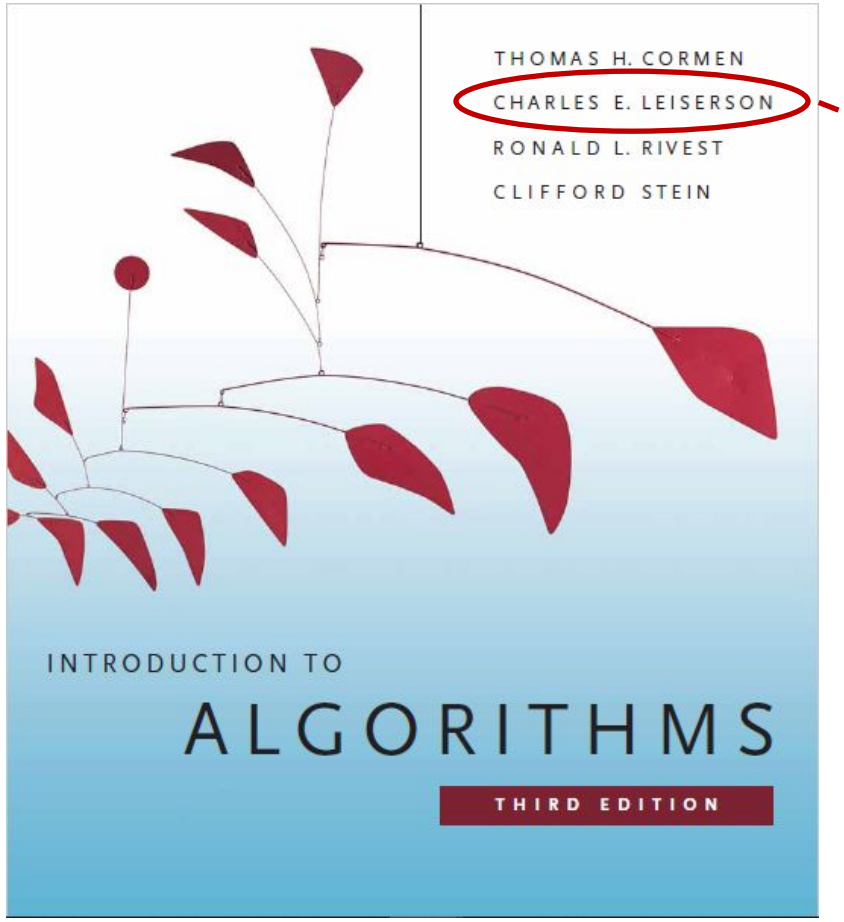
Contenido

- Preámbulo
  - Mecanismos de concurrencia con hilos en Java
- Estructura del artículo
- Introducción
- Un modelo de computación multihilos
- La propiedad de “hojas ocupadas”
- Un algoritmo de robo de trabajo (work-stealing) aleatorizado
- Acceso atómico y el juego de reciclaje
- Análisis del algoritmo de robo de trabajo (work-stealing)
- Otros artículos para revisar
- Conclusiones y recomendaciones

Preámbulo

- Blumofe y Leiserson presentaron la primera versión de un planificador con base en “robo de trabajo” (work stealing) aleatorizado (RWS) para computaciones multi-hilo con dependencias.
- Probaron los límites óptimos de los peores casos para
  - El tiempo de ejecución esperado,
  - El espacio de almacenamiento requerido
  - El costo total de la comunicación
- Además mostraron que el enfoque de *work stealing* tiene un costo menor en comunicaciones que *work sharing*.

¿Dónde lo vi mencionado por primera vez?



- *"Blumofe and Leiserson [52] gave a distributed scheduling algorithm for dynamic multithreading based on randomized "work-stealing" and showed that it achieves the bound"*

$$\mathrm{E}[T_P] \leq T_1/P + O(T_\infty)$$

- [52] Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748, 1999.

Estructura del artículo

- Abstract

1. Introduction
2. A model of multithreaded computation
3. The busy-leaves property
4. A randomized work-stealing algorithm
5. Atomic accesses and the recycling game
6. Analysis of the work stealing algorithm
7. Conclusion

- Acknowledgments
- References



Robert D. Blumofe



Charles E. Leiserson

Antes: revisión de los mecanismos de concurrencia con hilos en el lenguaje Java

¿En cuáles se utiliza el algoritmo de Work Stealing?

Mecanismos de concurrencia con hilos en Java



1. Synchronized
   - Java 1.0 [1996]
2. ReentrantLock
   - Java 5 (J2SE 5.0) [2004]
3. Executor/ExecutorService
   - Java 5 [2004]
4. ForkJoin
   - Java 7 [2011]
5. Virtual Threads
   - Java 21 (LTS) [2023]

1. **Synchronized [1996]**
   - Fue el mecanismo clásico y nativo de Java. Solo un hilo a la vez puede entrar a una sección crítica.
   - *Características principales:* Sintaxis integrada al lenguaje. Cerrojo automático al entrar y liberación automática al salir. Adecuado para exclusión mutua básica.
   - *Ventajas:* Muy simple. Menos propenso a olvidar liberar el lock. Bueno para casos pequeños y medianos.
   - *Deficiencias:* Poco flexible. No permite intentar adquirir el cerrojo con tiempos de espera de forma natural. No separa

Etapa en la que Java trataba la concurrencia como **protección de memoria compartida**.

Mecanismos de concurrencia con hilos en Java



2. ReentrantLock [2004]



- java.util.concurrent.locks es una familia de locks que incluye ReentrantLock.
- *Características principales*: Lock explícito con lock() y unlock(). Soporta tryLock(), tiempos de espera. Puede asociarse con variables de condición.
- *Ventajas*: Más flexible que synchronized. Permite estrategias como esperar cierto tiempo o abortar. Útil cuando hay varias condiciones de espera.
- *Deficiencias*: Si olvidas unlock(), se puede bloquear el sistema. Código más verboso. Aumenta complejidad conceptual.

Transición de concurrencia “embebida en el lenguaje” hacia concurrencia **más programable y configurable**.

3. Executor / ExecutorService [2004]

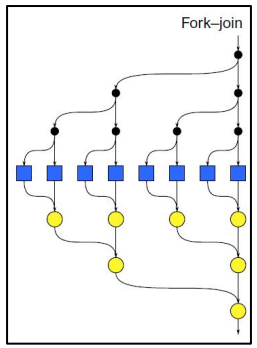
- Con Executor, Java separa el concepto de “qué trabajo hacer” del de “qué hilo lo ejecuta”.
- Características principales: Envío de tareas (Runnable, Callable) a un pool. Reutilización de hilos. Control del ciclo de vida del pool.
- *Ventajas*: Evita crear hilos manualmente para cada trabajo. Mejora uso de recursos. Facilita escalabilidad.
- *Deficiencias*: Aún depende de hilos del S.O. Elegir mal el tamaño del pool causa problemas.

Java cambia el enfoque de “manejar hilos” a **gestionar tareas concurrentes**.

Mecanismos de concurrencia con hilos en Java

4. ForkJoin [2011]

- Para trabajo fuertemente paralelo cuando puede dividirse recursivamente en subtareas más pequeñas y luego recombinar resultados.
- *Características principales:* Modelo divide-and-conquer. Uso de ForkJoinPool y Work Stealing [hilos ociosos roban trabajo a los otros hilos].
- *Ventajas*: Excelente para procesamiento paralelo intensivo con buen aprovechamiento de múltiples núcleos. Eficiente en algoritmos recursivos paralelos. Base de streams paralelos.
- Deficiencias: Más difícil de razonar que un Executor simple. Requiere buen diseño algorítmico.



Etapa donde Java madura hacia el **paralelismo estructurado y eficiente para CPU.**

5. Virtual Threads [2023]

- El proyecto Loom incorporó los *virtual threads* como respuesta a los hilos tradicionales -que son costosos- para tareas APIs o sistemas I/O-bound.
- *Características principales*: Hilos livianos gestionados por la JVM. Permiten tener cantidades masivas de tareas concurrentes.
- *Ventajas*: Escala mucho mejor que los hilos de plataforma para trabajo bloqueante. Acercan el estilo de programación a “un hilo por tarea”.
- *Deficiencias*: No elimina la necesidad de sincronización. Algunas operaciones pueden “anclar” el *virtual thread* a un *carrier thread*.

Java pasa de optimizar “pocos hilos caros” a soportar **muchas unidades concurrentes baratas.**

Mecanismos de concurrencia con hilos en Java

Modelo	Scheduler	Work stealing
synchronized	no	no
ReentrantLock	no	no
Executor	simple	no
ForkJoin	avanzado	sí
Virtual Threads	avanzado	sí (indirecto)

Equivalencias en otros lenguajes de los mecanismos de concurrencia con hilos en Java

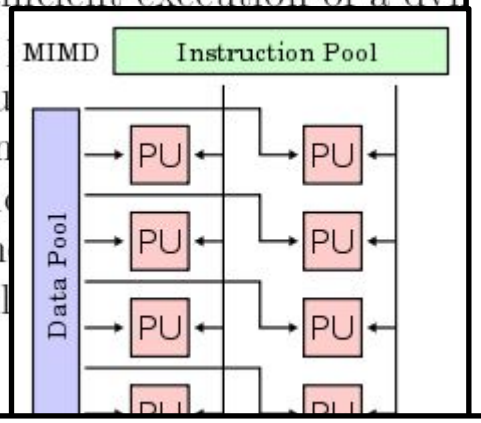
Java	Erlang	Go	Python
`synchronized`	no equivalente directo	sync.Mutex	`threading.Lock`
`ReentrantLock`	no típico	`sync.RWMutex` / patrones con canales	`threading.RLock`
`Executor`	procesos + mailbox + supervisors	`goroutines + worker pools`	`concurrent.futures.Executor`
`ForkJoin`	paralelismo por procesos, no equivalente exacto	`goroutines + scheduler + work stealing` interno	`multiprocessing.Pool`, `concurrent.futures.ProcessPoolExecutor`
`Virtual Threads`	procesos de Erlang	`goroutines`	no equivalente perfecto; `asyncio` es lo más cercano en espíritu, no en semántica

El enfoque de concurrencia es diferente. El lenguaje Erlang utiliza el modelo de actores, el lenguaje Go está fuertemente inspirado en CSP, en tanto que Java utiliza memoria compartida + hilos y Python usa hilos/procesos/async.

Iniciando con el artículo

Introducción

1 Introduction



En computación, **MIMD** (del inglés ***M**ultiple **I**nstruction, **M**ultiple **D**ata*, en español "múltiples instrucciones, múltiples datos") es una técnica empleada para lograr paralelismo. Las máquinas que usan MIMD tienen un número de procesadores que funcionan de manera asíncrona e independiente. En cualquier momento, cualquier diferentes datos.

Mantener los procesadores ocupados

Mantener los requerimientos de memoria dentro de límites adecuados

Minimizar la comunicación entre los procesadores

Introducción

Two scheduling paradigms have arisen to address the problem of scheduling multithreaded computations: ***work sharing*** and ***work stealing***. In work sharing, whenever a processor generates new threads, the scheduler attempts to migrate some of them to other processors in hopes of distributing the work to underutilized processors. In work stealing, however, underutilized processors take the initiative: they attempt to "steal" threads from other processors. Intuitively, the migration of threads occurs less frequently with work stealing than with work sharing, since when all processors have work to do, no threads are migrated by a work-stealing scheduler, but threads are always migrated by a work-sharing scheduler.

Dos paradigmas para planificación computaciones multi-hilo

1. **Work sharing**
2. **Work stealing**

Work sharing vs. Work stealing

Supervisor que "distribuye" el trabajo



Work sharing

NO hay Supervisor

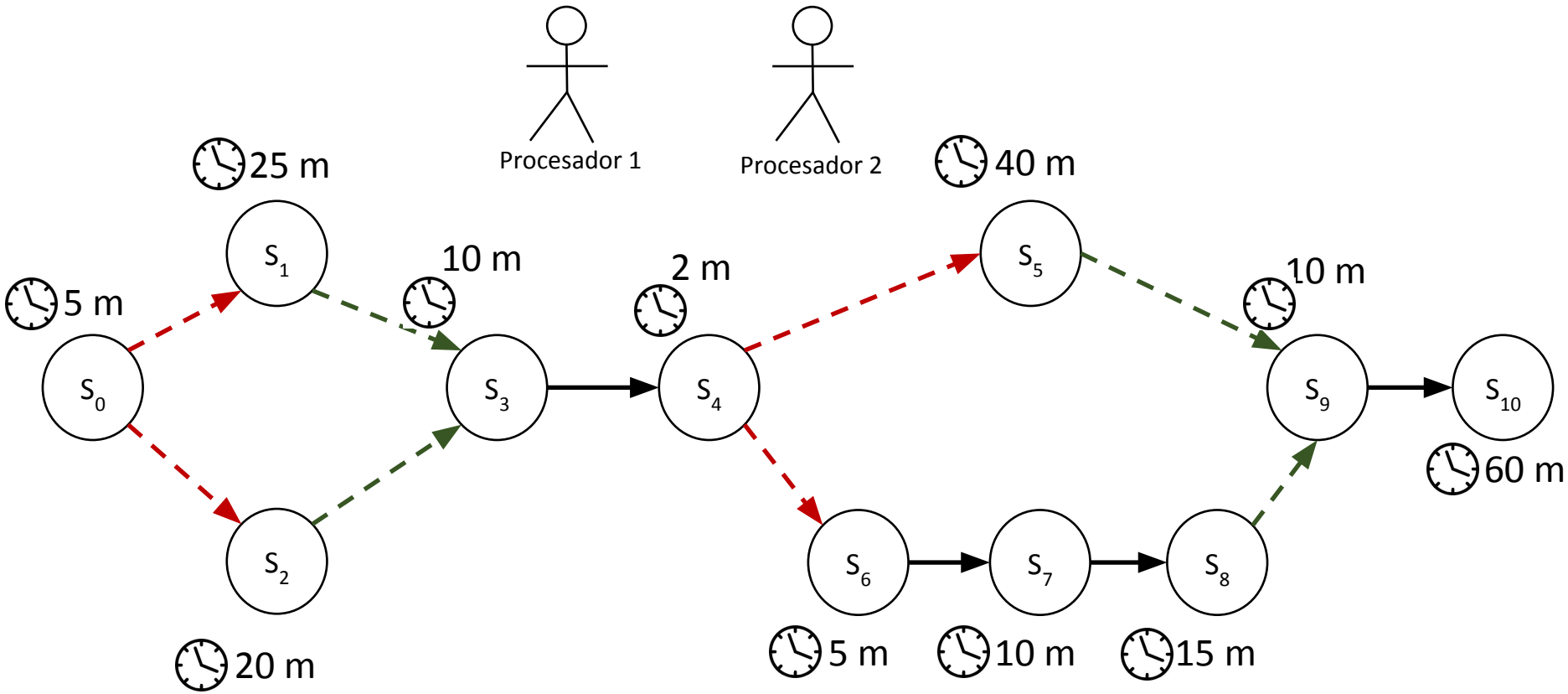


Work stealing

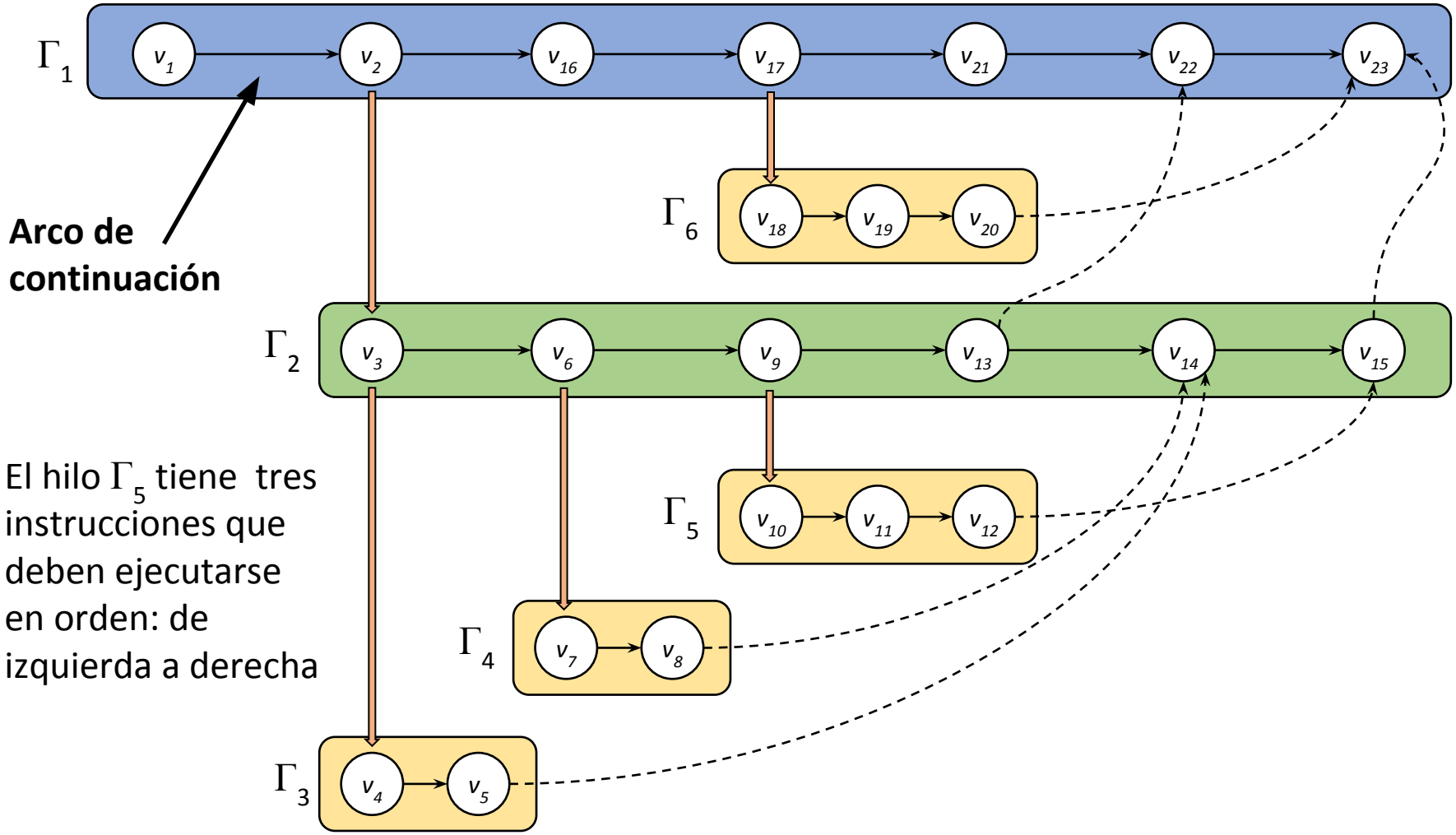
Un modelo de computación multihilos

- Una computación multihilos está compuesta de un conjunto de hilos, cada uno de los cuales es un ordenamiento secuencial de instrucciones. En el modelo, cada instrucción utiliza una unidad de tiempo para su ejecución.
- La instrucciones están conectadas por arcos de dependencia (*dependency edges*) que proporcionan un orden parcial en el cual las instrucciones deben ser ejecutadas.

Grafo computacional



Computación multihilo con 6 hilos [los Γ_i] y 23 instrucciones [los círculos]

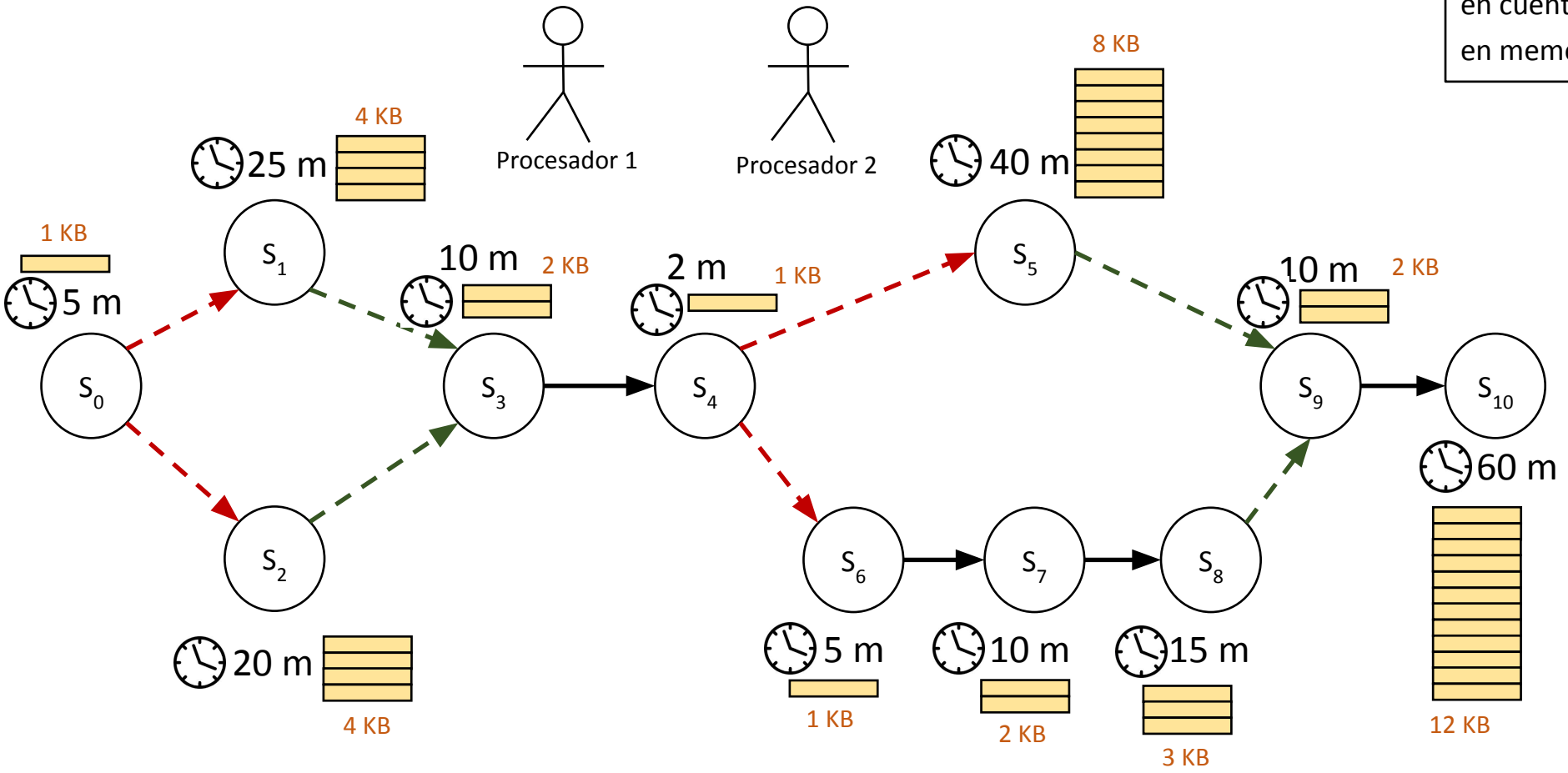


Activation frame

- Para ejecutar un hilo, a este se le asigna una porción de memoria que se llamará "*activation frame*"
  - Es un espacio de memoria que pueden utilizar las instrucciones del hilo para almacenar los valores que está procesando.

Grafo computacional con *activation frames*

NOTA: esta gráfica también es conceptual y ayuda a recordar que en el procesamiento paralelo se debe tener en cuenta el espacio en memoria.



Tamaño de pila

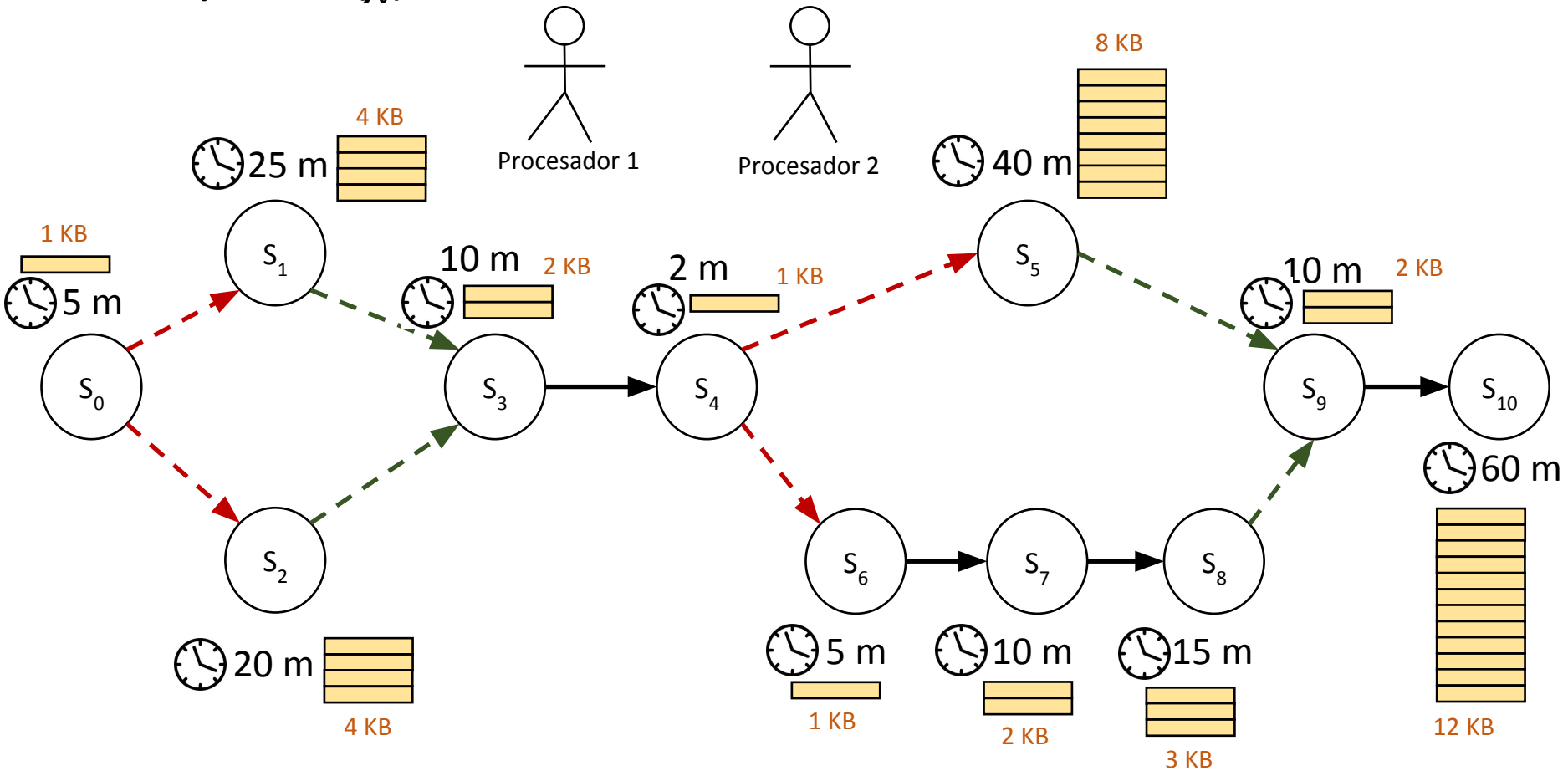
- El **tamaño de la pila (*stack depth*) de un hilo** es la suma de los tamaños de los “activation frames” de todos sus ancestros, incluyéndose él mismo.
- El **tamaño de la pila (*stack depth*) de una computación** multihilo es el **máximo tamaño de pila** entre los hilos que conforman la computación

Espacio de memoria

- El espacio de memoria utilizado por una computación multihilo con P procesadores, con la planificación de ejecución χ, se denota por $S(\chi)$.
- El espacio utilizado por una computación multihilo con 1 procesador, se denota por S_1.
- Una planificación de ejecución se dice que exhibe una expansión lineal del espacio cuando $S(\chi) = O(S_1 P)$ y se considera óptima

Grafo computacional y Espacio de memoria

Espacio $S(\chi) = 1 + 4 + 4 + 2 + 1 + 8 + 1 + 2 + 3 + 2 + 12 = 40$



Un repaso de complejidad asintótica

Complejidad asintótica

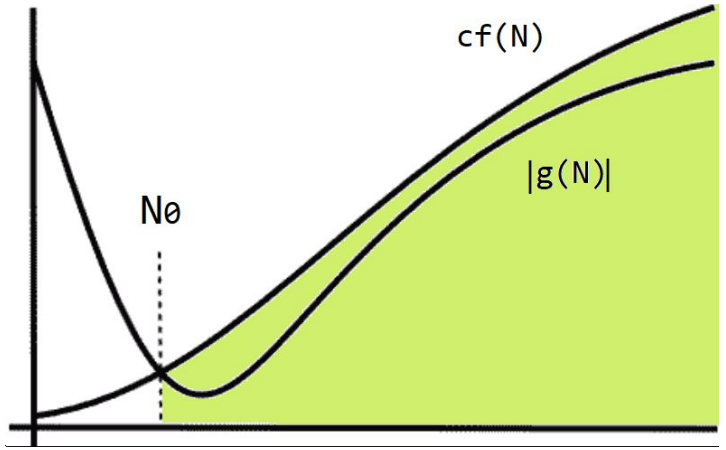
- Se utiliza para comparar el comportamiento de los algoritmos independiente del hardware donde podría ejecutarse.
- En una computación serial, la **complejidad en el tiempo** de un algoritmo resume cómo el tiempo de ejecución del algoritmo se incrementa con el tamaño de la entrada [cantidad de datos que procesa el algoritmo]
- La **complejidad en el espacio** resume la cantidad de memoria que un algoritmo requiere a medida que se incrementa el tamaño de la entrada.
- Estas dos medidas de complejidad **ignoran los factores constantes** ya que estos dependen de las características de la máquina como el conjunto de instrucciones de máquina o la rapidez del reloj.
- Por el contrario, estas medidas se enfocan en el crecimiento asintótico, que es independiente del hardware en particular, permitiendo la comparación de diferentes algoritmos sin importar el computador real que se utilice.
- Para entradas de tamaño suficientemente grandes, los efectos asintóticos dominarán cualquier ventaja debida a un factor constante.

Complejidad asintótica

- La complejidad en el tiempo calculada con base en el modelo work-span no permite predecir de manera completa el desempeño del algoritmo.
  - El modelo work-span solo considera la computación, no considera los efectos de la comunicación entre los núcleos ni del uso de la memoria.
- La complejidad asintótica es un indicador del desempeño en tamaños de problemas suficiente mente grandes y revela los límites fundamentales del algoritmo.

Notación O grande (big O notation)

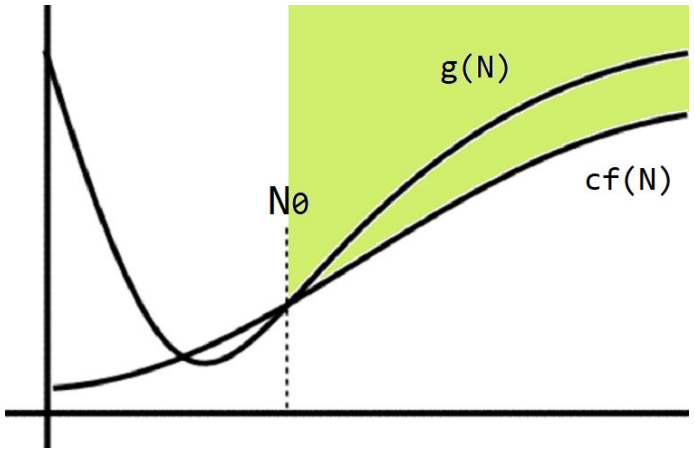
- Describe un conjunto de funciones con una cota o límite superior.



O(f(N)) es el conjunto de todas las funciones g(N) tales que existen las constantes positivas c y N_0 que satisfacen $|g(N)| \leq cf(N)$ para $N \geq N_0$

Notación Omega grande (big Omega notation)

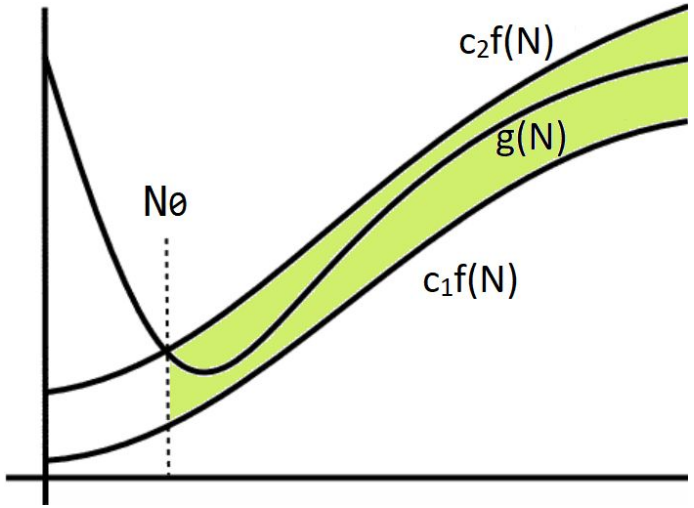
- Describe un conjunto de funciones con una cota o límite inferior.



$\Omega(f(N))$ es el conjunto de todas las funciones g(N) tales que existen las constantes c y N_0 que satisfacen $g(N) \geq cf(N)$ para $N \geq N_0$

Notación Theta grande (big Theta notation)

- Describe un conjunto de funciones con una cota o límite superior y una cota o límite inferior.



$\Theta(f(N))$ es el conjunto de todas las funciones g(N) tales que existen las constantes positivas c_1, c_2 y N_0 que satisfacen $c_1 f(N) \leq g(N) \leq c_2 f(N)$ para $N \geq N_0$

Complejidad asintótica

- En la notación utilizada para la expresar resultados de complejidad asintótica se abusa del significado del signo “=“. Dependiendo del contexto puede significar pertenencia o inclusión.

$T(N) = O(f(N))$ Significa ? $T(N) \in O(f(N))$

$O(f(N)) = O(h(N))$ Significa ? $O(f(N)) \subseteq O(h(N))$

$T(N) = O(N^2) = O(N^3)$ Significa ? $T(N) \in O(N^2) \subseteq O(N^3)$

?

Resumen de notaciones asintóticas.

Notación	Descripción	Definición
$f(n) = O(g(n))$	f escala $\leq g$	$\exists\, c, n_0 \ni f(n) \leq cg(n)\ \forall\, n > n_0$
$f(n) = o(g(n))$	f escala $< g$	$\forall\, c > 0\ \exists\, n_0 \ni f(n) < cg(n)\ \forall\, n > n_0$
$f(n) = \Omega(g(n))$	f escala $\geq g$	$\exists\, c, n_0 \ni f(n) \geq cg(n)\ \forall\, n > n_0$
$f(n) = \omega(g(n))$	f escala $> g$	$\forall\, c > 0\ \exists\, n_0 \ni f(n) > cg(n)\ \forall\, n > n_0$
$f(n) = \Theta(g(n))$	f escala $= g$	$\exists\, c_1, c_2, n_0 \ni c_1 g(n) \leq f(n) \leq c_2 g(n)\ \forall\, n > n_0$

El símbolo matemático $\exists$ significa «existe», $\ni$ significa «tal que» y $\forall$ significa «para todos».

Complejidad asintótica

- En el análisis asintótico de programas seriales, es común utilizar el análisis “O grande”, [big O] debido que se suele encontrar la cota superior del tiempo o el espacio del programa.
- Para programas paralelos, el análisis “Θ” grande generalmente es más útil ya que se requiere encontrar los limites entre los que se encuentra una razón (la comparación entre dos componentes), como el speedup [mejora en la rapidez]. En este caso se requiere estimar una cota inferior para el numerador y una cota superior para el denominador.

Complejidad asintótica

Speedup =< T_1/T_∞

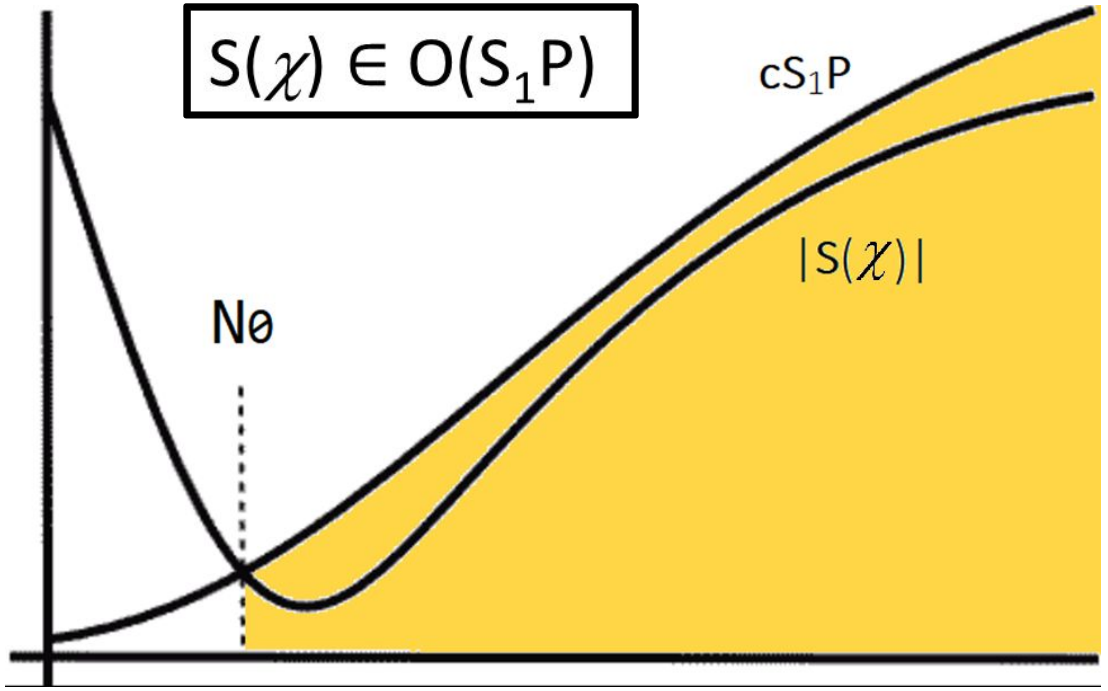
- Ejemplo:
  - Necesitamos mostrar que utilizando P hilos se logra que un algoritmo paralelo se ejecute $\sqrt{P}$ más rápido que su versión serial.
- Es decir se requiere una cota inferior ("Ω") para el speedup. Esto requiere encontrar
  - Una cota inferior ("Ω") para el tiempo del algoritmo serial y
  - Una cota superior ("O") para el tiempo de la versión paralela del algoritmo.
- Cuando se calcula el speedup, el tiempo de la versión serial estará en el numerador y el tiempo de la versión paralela estará en el denominador.
  - Un valor de tiempo grande en la versión paralela reducirá el valor de speedup
  - Un valor de tiempo grande en la versión serial incrementará el valor de speedup.
- Conceptualmente, es más fácil trabajar con la estimación de omega grande ("Θ").

De regreso al artículo

Qué nos dice $S(\chi) = O(S_1P)$

- Una planificación de ejecución se dice que exhibe una expansión lineal del espacio cuando $S(\chi) = O(S_1P)$ y se considera óptima

Los valores de la función que calcula el espacio de memoria utilizado por una computación multihilo para P procesadores, con la planificación de ejecución X, forma parte de un conjunto de funciones, que a partir de una constante N_0 está por debajo de cS_1P



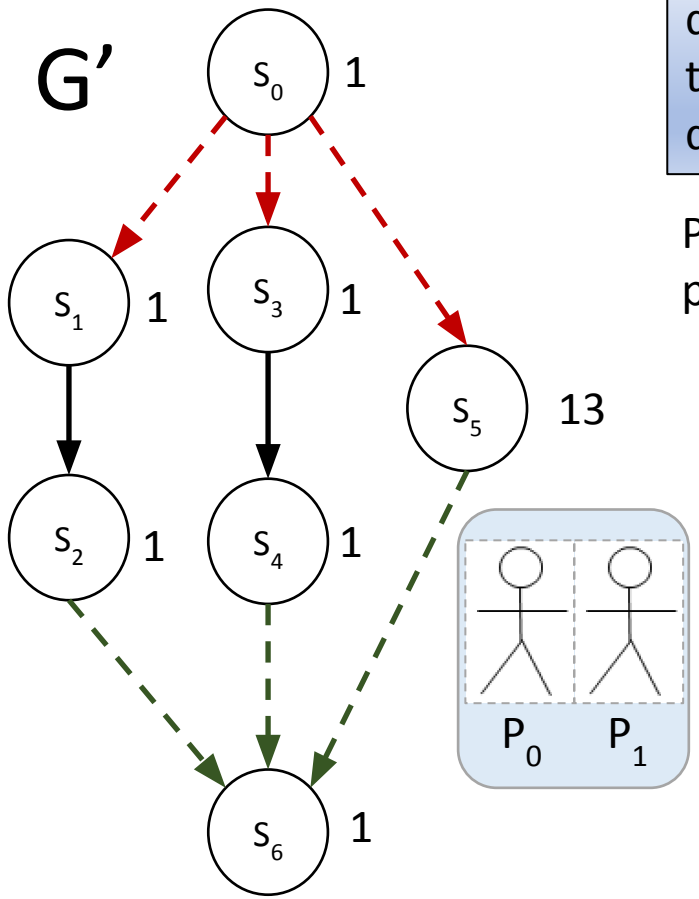
Planificación de ejecución para P procesadores

- La planificación de ejecución para P procesadores (*P-processor execution schedule*) en una computación multihilo determina qué procesadores de un computador paralelo con P procesadores ejecutarán qué instrucciones en cada paso de la computación.
  - Depende de la computación en particular y la cantidad P de procesadores
- En cualquier paso dado de una planificación de ejecución, cada procesador ejecuta al menos una instrucción.

Planificación (scheduling) en multiprocesadores: ejemplo 1

Si el procesamiento se realiza en un computador que tiene *p* procesadores, se dice que T_p es el tiempo transcurrido entre el inicio y la finalización del procesamiento.

Para este ejemplo, se tiene un computador con P = 2 procesadores: P_0 y P_1.

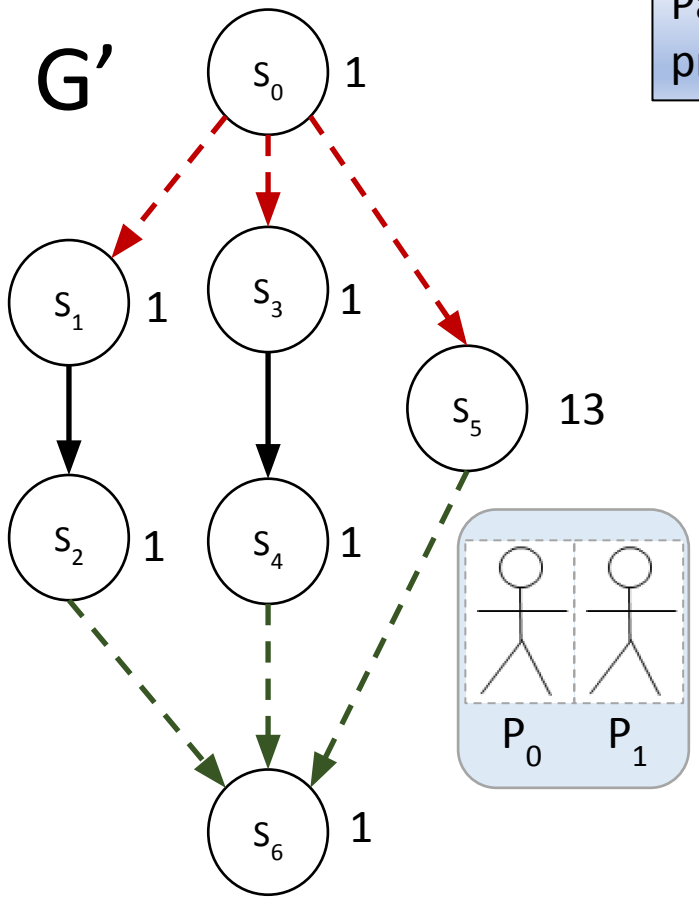


Scheduling No. 1 con P = 2

T_2 = 17

P_0	P_1
$S_0[1]$	Ocioso
$S_1[1]$	$S_3[1]$
$S_2[1]$	$S_4[1]$
$S_5[13]$	**Ocioso [13]**
$S_6[1]$	Ocioso

Planificación (scheduling) en multiprocesadores: ejemplo 2



Para este ejemplo, se tiene un computador con P = 2 procesadores: P_0 y P_1.

Scheduling No. 2 con P = 2
Priorizando S_5

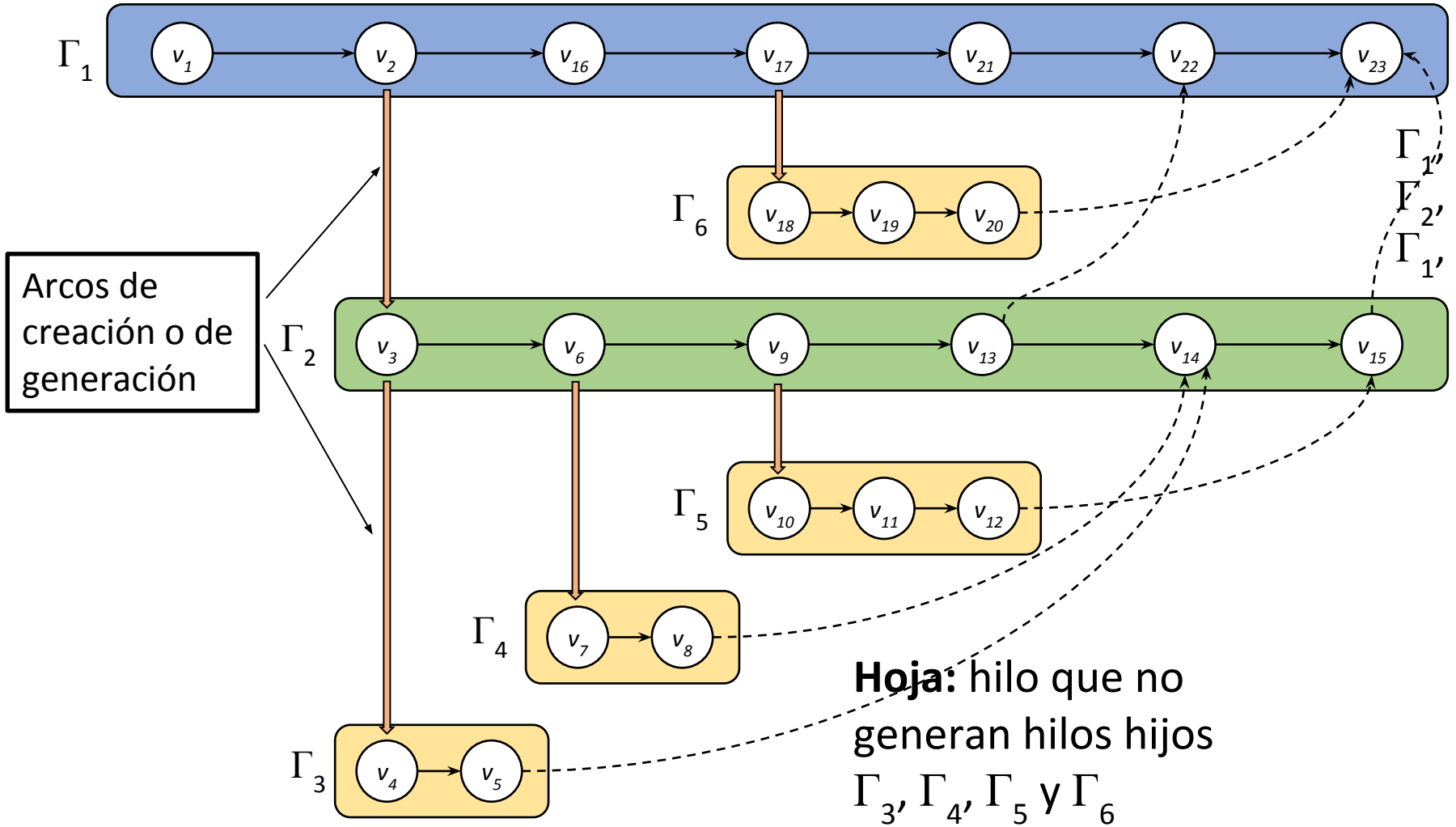
P_0	P_1
$S_0[1]$	Ocioso
$S_5[13]$	$S_1[1]$
	$S_2[1]$
	$S_3[1]$
	$S_4[1]$
	Ocioso [9]
$S_6[1]$	Ocioso

T_2 = 15

Árbol de generación [creación]

- Durante la ejecución de una computación, un hilo puede crear o generar (*spawn*) otros hilos.
- Generar un nuevo hilo se puede parecer al llamado a una subrutina en un programa, con la diferencia que el hilo generado puede ejecutarse concurrentemente con el hilo que lo generó.
- Aquí se considerará que los hilos generados son hijos del hilo que los generó y un hilo puede generar tantos hilos como desee.
- En consecuencia los hilos pueden representarse en un árbol de generación (*spawn tree*)

Árbol de generación



Hilos vivos, hilos que fallecen

- Cada arco de creación va de una instrucción específica, la que hace la operación de creación, a la primera instrucción del hilo hijo.
  - Un procesador NO puede iniciar la ejecución de un hilo hijo hasta que la instrucción del padre que crea el hijo haya sido ejecutada.
  - En la computación de ejemplo, en el arco (v_6,v_7), v_7 no puede ejecutarse solo después de v_6.
- Cuando la instrucción de creación se ejecuta, esta asigna un "*activation frame*" para el nuevo hijo.
  - Una vez un hilo ha sido creado y se le ha asignado un "*activation frame*", decimos que ese hilo está vivo "*alive*")
  - Cuando la última instrucción de un hilo se ejecuta y se desasigna el "*activation frame*" al hilo, decimos que el hilo ha muerto (*the thread dies*)

Arcos de juntura

- Además de los arcos de continuación y los arcos de creación se tienen los arcos de juntura (*join edges*)
- Estos se utilizan para representar instrucciones que producen datos para ser consumidos por otra instrucción (modelo productor consumidor)
- La relación productor consumidor exige que la instrucción que consume solo pueda ser ejecutada después que haya terminado la instrucción que produce
- Los arcos de juntura en el ejemplo son representados utilizando lineas curvas a trazos.

Hilos postergados, hilos “*ready*”

- Si una instrucción que consume llega a su punto de inicio y no ha terminado las instrucciones el hilo que produce los valores no ha terminado la ejecución, el hilo que consume posterga (*threat stalls*) su ejecución
- Cuando el hilo que produce los valores se ejecuta, se dice que el arco de dependencia se resuelve y habilita al hilo que consume para que reanude su ejecución. Decimos que el hilo está listo (*thread ready*)

Supuestos con los arcos de unión

- Hay dos supuestos con los arcos de juntura:
  - Se asume que cada instrucción tiene como máximo un número constante de arcos de unión incidentes (no es variable). Modelo de tiempo unitario para las instrucciones.
  - Se asume que ningún arco de unión entra en la instrucción que sigue inmediatamente una creación: cuando el hilo padre crea un hijo, el padre no puede quedar en estado de postergación inmediata.
    - Permanece en estado ready por lo menos una instrucción más.

dag

- Una planificación de ejecución (*execution schedule*) debe respetar las restricciones indicadas por los arcos de continuación, creación y de unión.
- Estos arcos de dependencia forman un grafo dirigido de instrucciones y ningún procesador puede ejecutar una instrucción hasta después que todas las instrucciones que le preceden, en este grafo, hayan sido ejecutadas.
- Si la planificación de ejecución existe, el grafo debe ser acíclico.
- Debe ser un grafo acíclico dirigido: dag (en inglés)
- En cualquier paso de una planificación de ejecución, una instrucción está en estado ready si todas las instrucciones que le preceden en el dag han sido ejecutadas

Suspuestos con los hilos padre

- En este modelo se asumirá que los hilos padres permanecerán vivos hasta que todos sus hilos hijo mueran.
  - Por esta razón un hilo padre no desasignará su *activation frame* hasta que todos los frames de sus hijos hayan sido desasignados. (esto no es absolutamente necesario, pero simplificará el análisis de la utilización del espacio de memoria)
  - Tampoco existirá memoria global por fuera de los *activation frames* asignados a cada hilo. O si existe, NO se tendrá en cuenta.
- Por tanto, el espacio utilizado en un momento dado al ejecutar una computación es el tamaño total de todos los frames utilizados por todos los hilos vivos en ese instante.
- Y el espacio total utilizado en la ejecución de una computación es el máximo de tales valores durante el transcurso de la ejecución.

Computación multihilo (resumen hasta aquí)

- Una computación multihilo puede ser vista como
  - Un dag de instrucciones conectado por arcos de dependencia.
  - Las instrucciones están conectadas por arcos de continuación dentro de los hilos.
  - Los hilos forman un árbol de generación/creación con los arcos de creación.
  - Cuando un hilo es creado, un *activation frame* es asignado y este frame permanece asignado hasta que el hilo permanezca vivo.
  - Un hilo vivo puede estar en estado *ready* o en estado postergado debido a que no se ha resuelto una dependencia.



Referencia principal

- Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748, 1999.

Gracias